

COS20019 – CLOUD COMPUTING ARCHITECTURE

Assignment 2: Developing a highly available Photo Album website

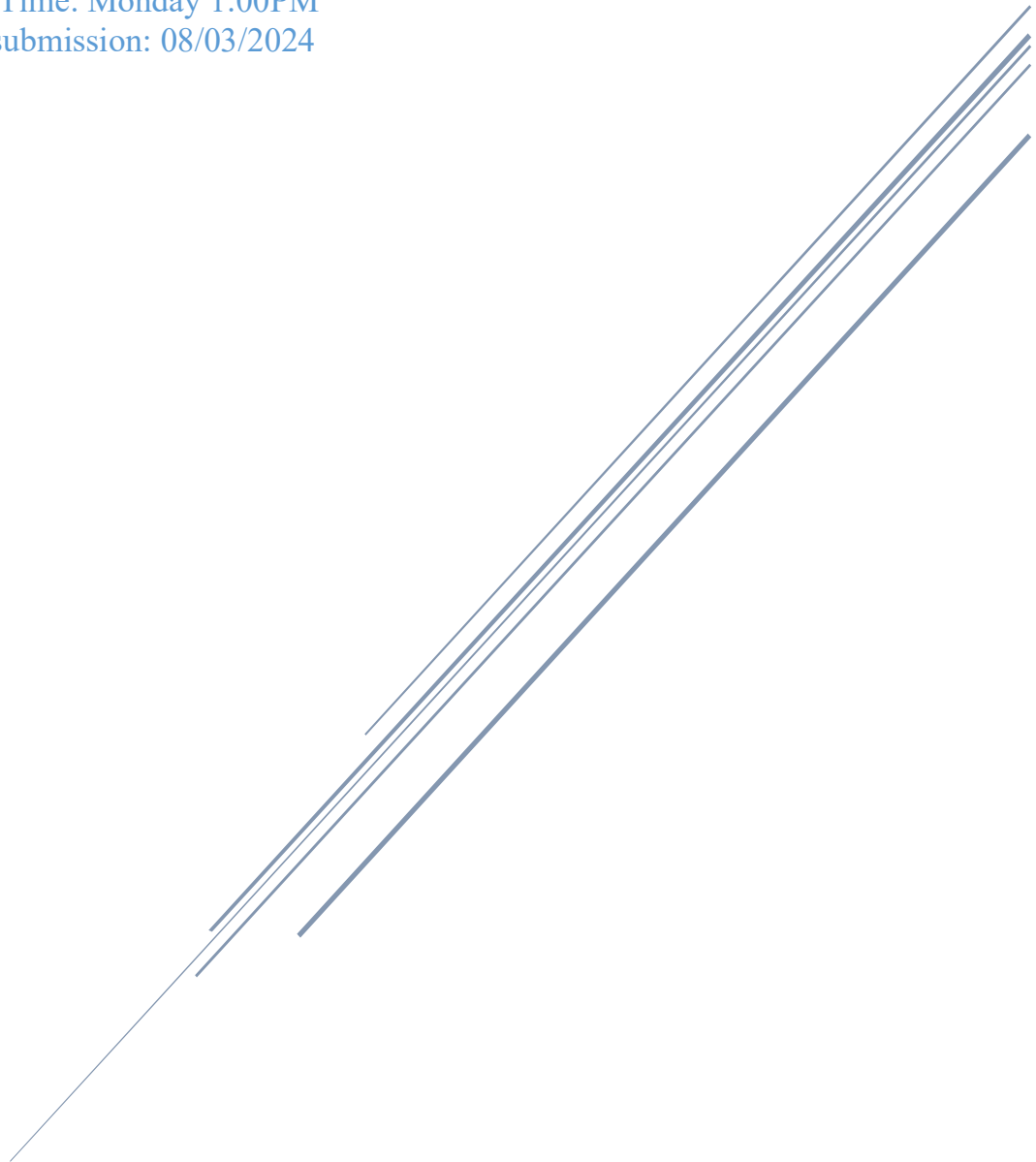
Student Name: Nguyen Thi Thanh Minh

Student ID: 104169617

Tutor: Dr. Tran Minh Hai

Tutorial Time: Monday 1:00PM

Date of submission: 08/03/2024



I. INTRODUCTION

In this assignment, we extended the web application from Assignment 1B to make it highly available. We used an IAM role to enable interactions between EC2, Lambda and S3. We created and limited access to an S3 bucket using Bucket Policy, created a Lambda function, a Dev Server instance, a custom AMI, a Launch Template, an Elastic Application Load Balancer (ALB) and an Auto Scaling group (ASG) across 2 Availability Zones (AZs) for efficient high availability of the photo album web servers. VPC, security groups and RDS database were also configured properly.

Additionally, we set up an EC2 NAT instance instead of a NAT gateway. We also modified the Target group so that phpMyAdmin could be accessed from servers in private subnets. We configured a Network ACL for private subnets to block bidirectional ICMP traffic to/from Dev Server. Here's the link to the website (DNS name of the ALB because **AWS does not support Elastic IP address to be associated to an ALB**):

<http://photoalbum-elb-336195294.us-east-1.elb.amazonaws.com/photoalbum/album.php>

II. INFRASTRUCTURE DEPLOYMENT

A. VPC

The VPC **MNguyenVPC** from Assignment 1B was reused. It had 4 subnets – 2 public, 2 private – in 2 AZs us-east-1a and us-east-1b, each zone having 1 public subnet and 1 private subnet (Figure 1, 2).

Public subnets were associated with the Public Route Table which had a route to an Internet Gateway (Figure 1), while private subnets were attached to the Private Route Table and had its internet traffic routed to the network interface of an EC2 instance called **NAT Server** (Figure 3). This was launched using the specified AMI (Figure 4) and had its Source/destination check disabled, about which we will go into details later in this report.

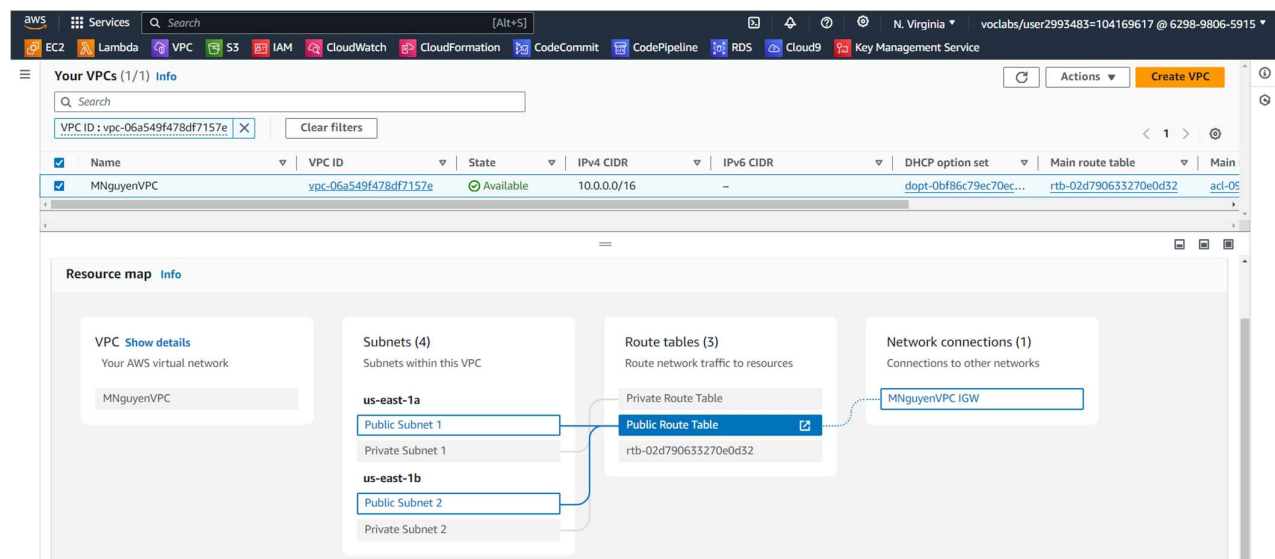


Figure 1 – MNguyenVPC with resource map, including 4 subnets in 2 AZs, 2 route tables and 1 internet gateway

Name	Subnet ID	State	VPC	IPv4 CIDR	IPv6...	Available IPv4...	Availability Zone	Availability Zone ID
Public Subnet 2	subnet-0d6777d89d8bba47d	Available	vpc-06a549f478df7157e MNguyenVPC	10.0.2.0/24	–	249	us-east-1b	use1-az2
Private Subnet 2	subnet-0c941ed0341adad2	Available	vpc-06a549f478df7157e MNguyenVPC	10.0.4.0/24	–	250	us-east-1b	use1-az2
Public Subnet 1	subnet-0fbca2a68f3a9e833	Available	vpc-06a549f478df7157e MNguyenVPC	10.0.1.0/24	–	249	us-east-1a	use1-az1
Private Subnet 1	subnet-0cf0b42e73827cc8d	Available	vpc-06a549f478df7157e MNguyenVPC	10.0.3.0/24	–	249	us-east-1a	use1-az1

Figure 2 – Details of subnets in MNguyenVPC

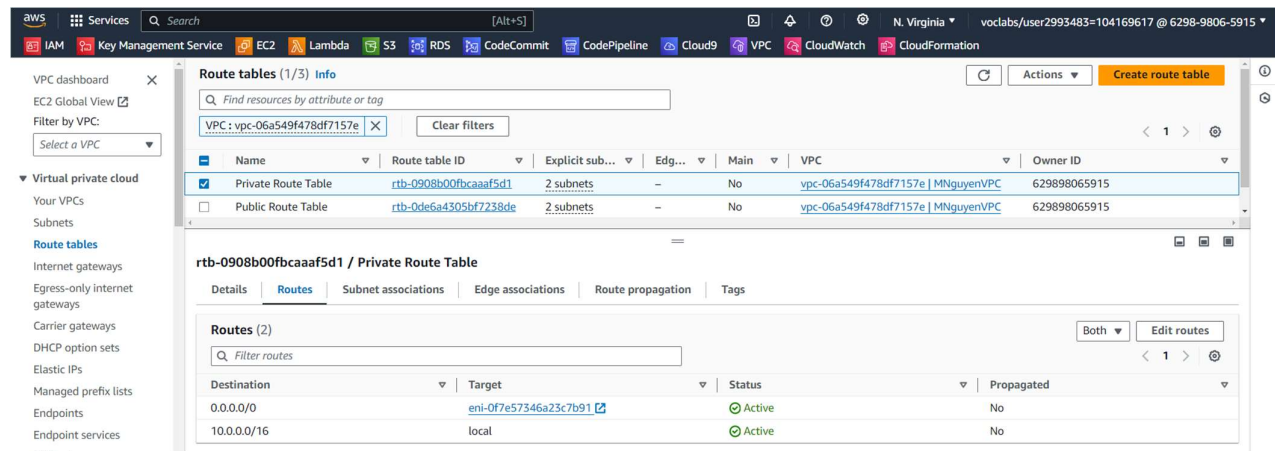


Figure 3 – Private Route Table had a route to the network interface of NAT Server EC2 instance

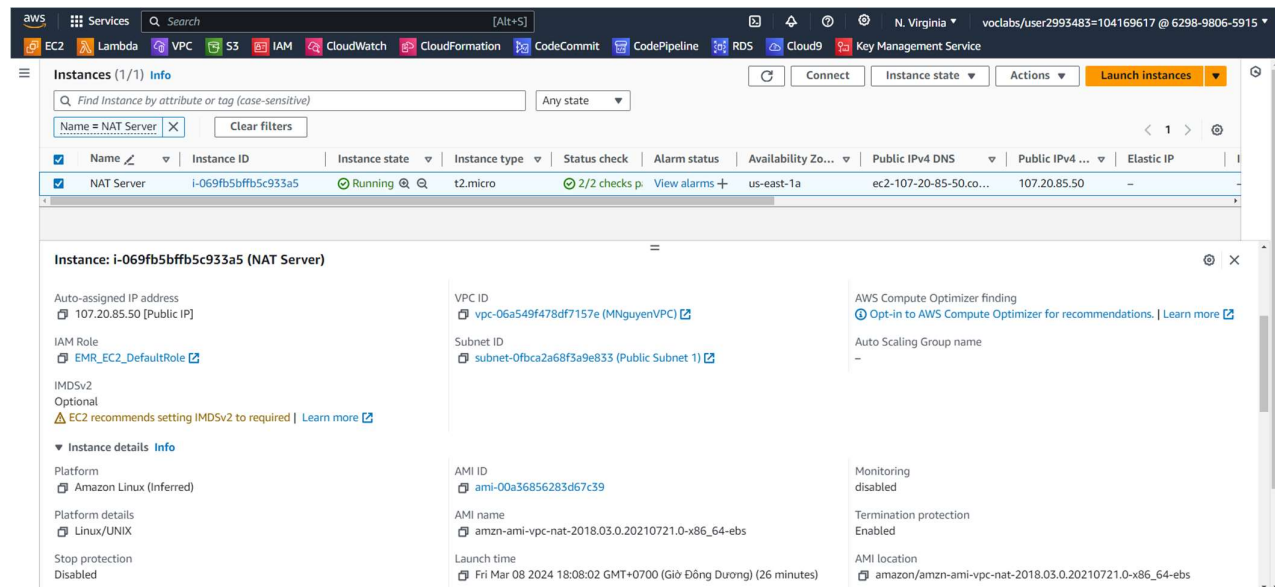


Figure 4 – The NAT Server instance running required AMI in Public Subnet 1

B. S3 photo storage

We created an S3 bucket similarly to one from the previous assignment. Then, we configured the bucket to block some public access (but not all). Specifically, the bucket blocks public access through access control lists (ACLs), but allows that through bucket policy (Figure 5).

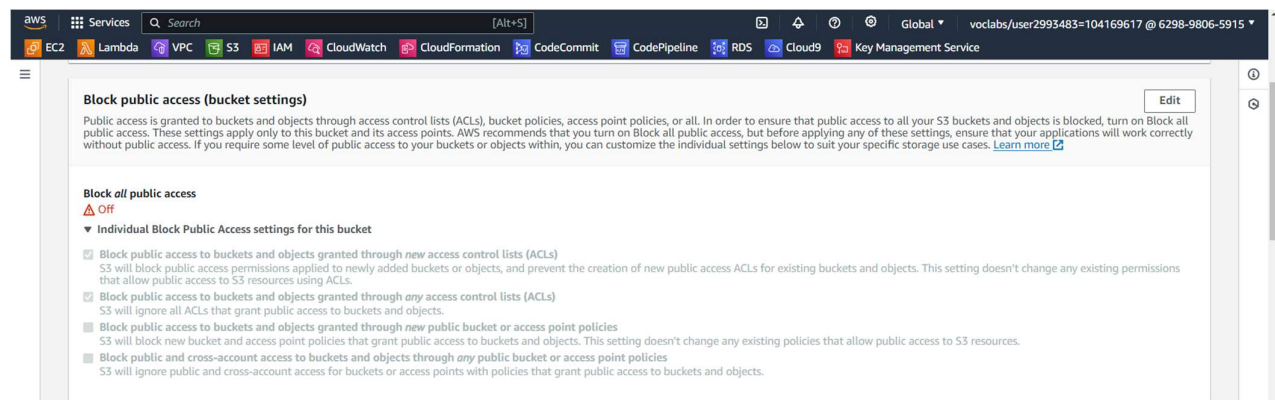


Figure 5 – Turning on some Block Public Access settings for the S3 bucket

As for bucket policy, we defined 2 statements to allow read action from a specific HTTP referrer which was actually the DNS name of the Application Load Balancer (ALB) for the Web Servers, and to allow read and upload from actors with the IAM role **LabRole** (Figure 6) [1].

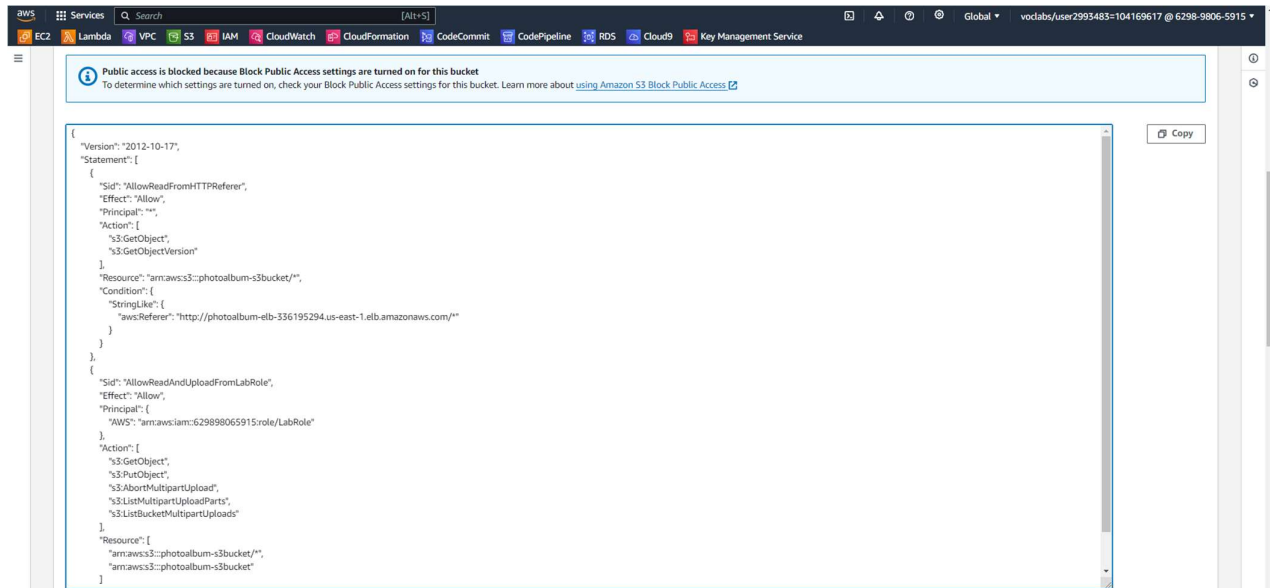


Figure 6 – S3 bucket policy to make the objects only accessible from web server or AWS resources with LabRole

C. Load balancing

In this step, we first created a target group named **WebServer** in **MNguyenVPC** that allowed health checks with required path `/photoalbum/album.php` (Figure 7). The health checks would be performed by accessing the `/var/www/html/photoalbum/album.php` path on each instance.

Additionally, in the **Attributes** tab of the target group, we turned on **Stickiness** with a duration of 5 minutes (Figure 8). This allowed persistent cookie sessions and ensured that requests from the same client go to the same target during the stickiness duration, which made phpMyAdmin accessible from servers in the Auto Scaling group in private subnets. By default, ALBs distribute incoming requests across instances based on their current load, which might lead to a lack of session persistence [2].

Next, we created an Internet-facing ALB **Photoalbum-ELB** mapped to public subnets of the VPC (Figure 9), with an HTTP:80 Listener forwarded to **WebServer** target group (Figure 10) and with security group **ELBSG** (Figure 11).

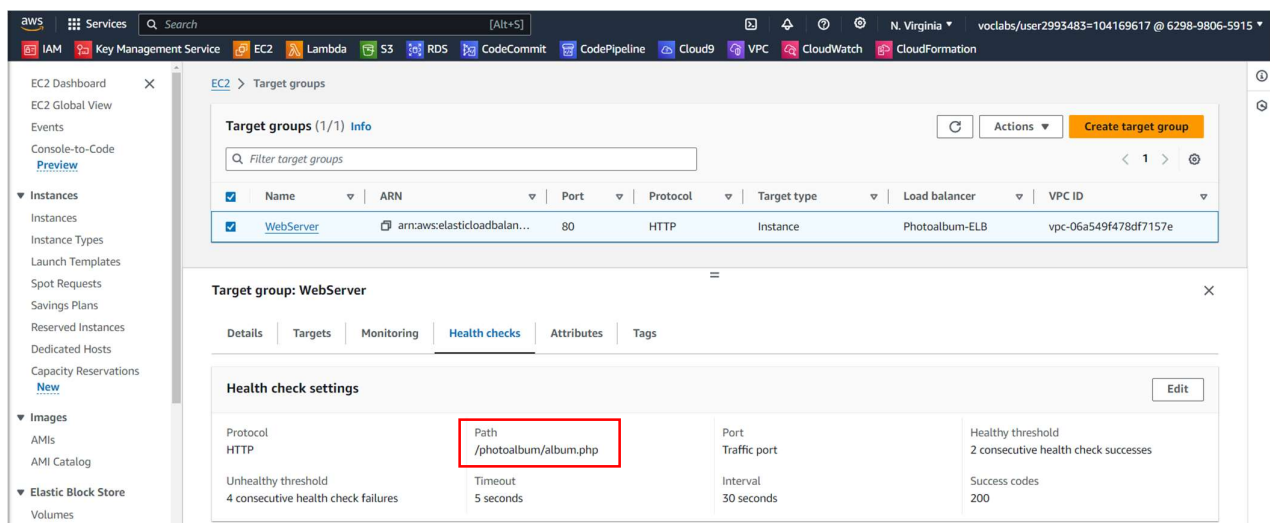


Figure 7 – Target group “WebServer” had the health check path as `/photoalbum/album.php`

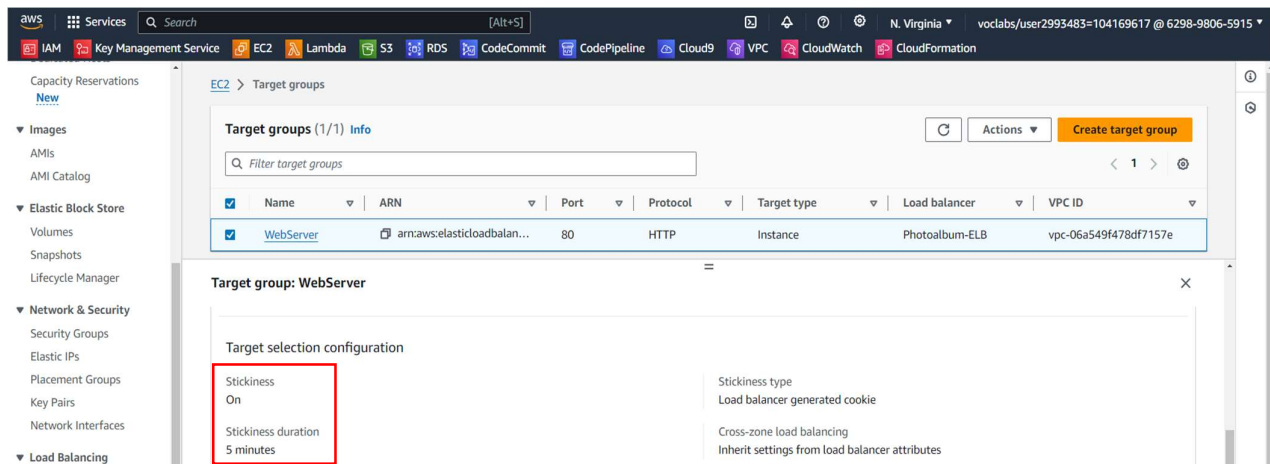


Figure 8 – Enabling duration-based stickiness for the target group

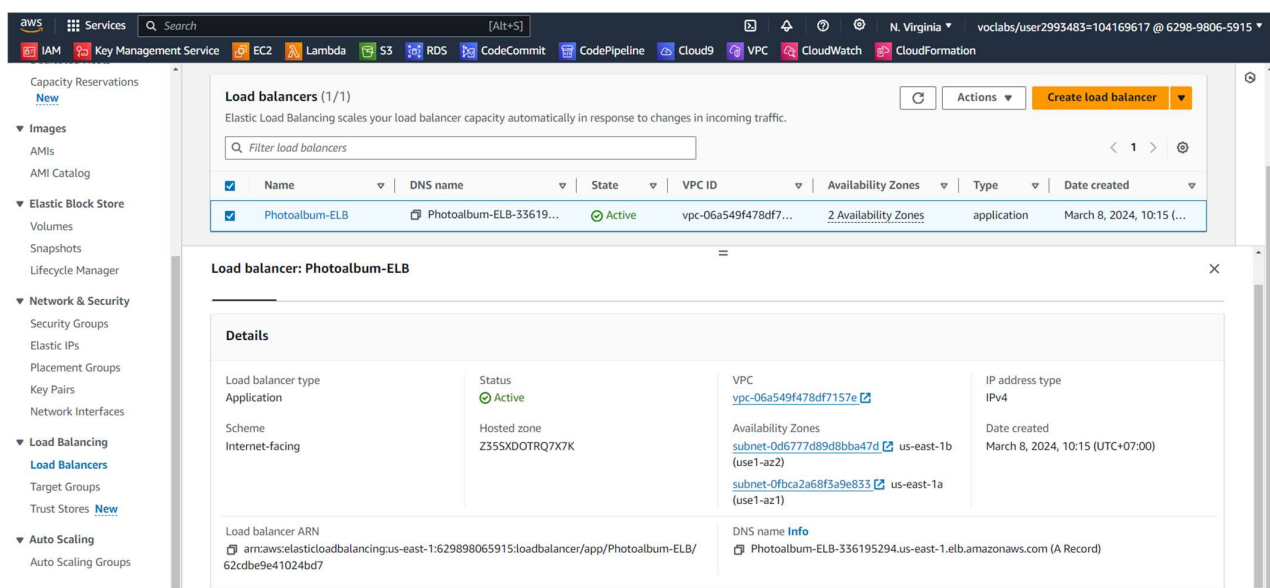


Figure 9 – Application Load Balancer “Photoalbum-ELB” created in public subnets of MNguyenVPC

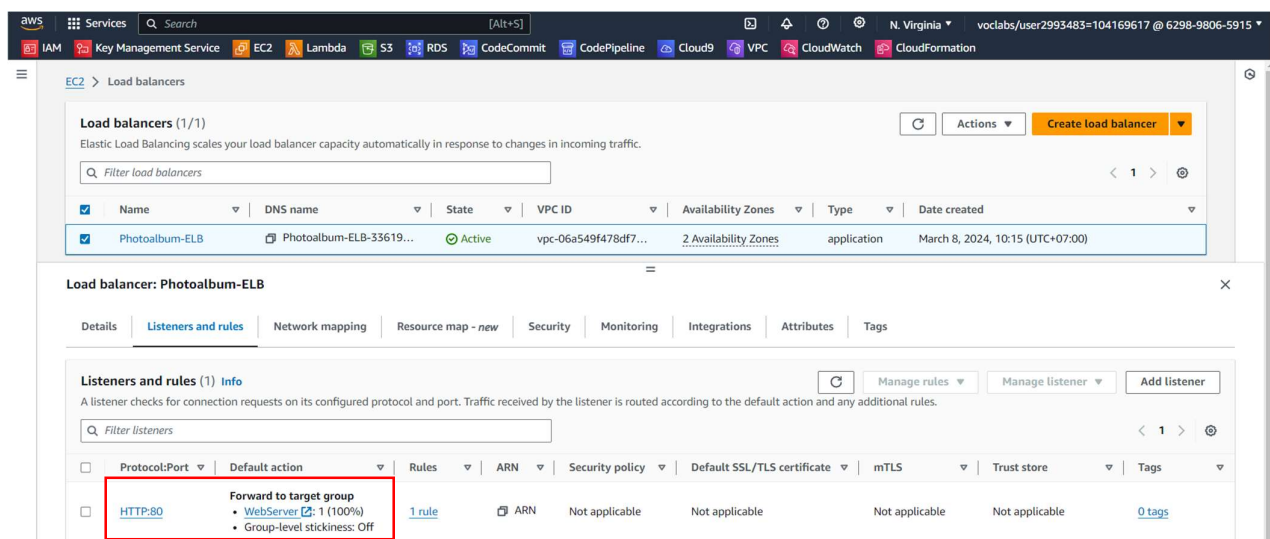


Figure 10 – “Photoalbum-ELB” had a listener for HTTP traffic on port 80 to forward to target group “WebServer”

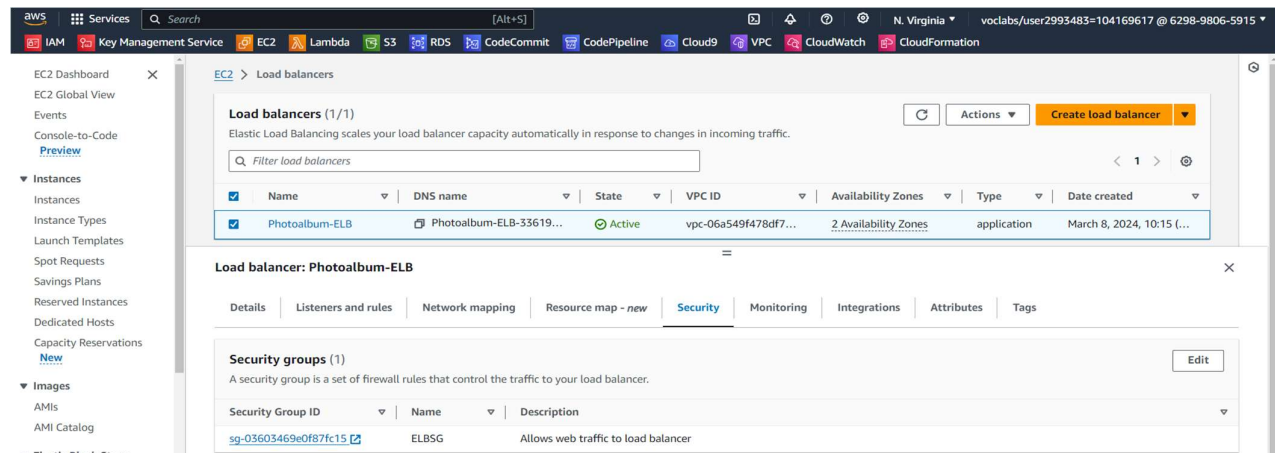


Figure 11 – The ALB belonged to the “ELBSG” security group

D. Network Address Translation (NAT)

As briefly stated before, we utilized an EC2 instance to function as a NAT Server that allowed private instances to communicate with the public internet. According to the requirements, we needed to use a specific AMI with ID `ami-00a36856283d67c39`, which could not be browsed on the AWS console because it was an End of Life version of Amazon Linux. Thus, we had to use the CloudShell CLI to launch the instance by specifying the AMI ID, instance type (`t2.micro`), key name, security group ID of NATServerSG and subnet ID of Public Subnet 1 (Figure 12).

After launch, we configured the instance according to the Amazon Virtual Private Cloud User Guide for NAT instances [3]. There were some commands incompatible with the OS, but we managed to setup **NAT Server** successfully with these commands:

```
sudo yum update -y
sudo yum install iptables -y
sudo service iptables start
sudo chkconfig iptables on
echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/custom-ip-forwarding.conf
sudo sysctl -p /etc/sysctl.d/custom-ip-forwarding.conf
sudo /sbin/iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
sudo /sbin/iptables -F FORWARD
sudo service iptables save
```

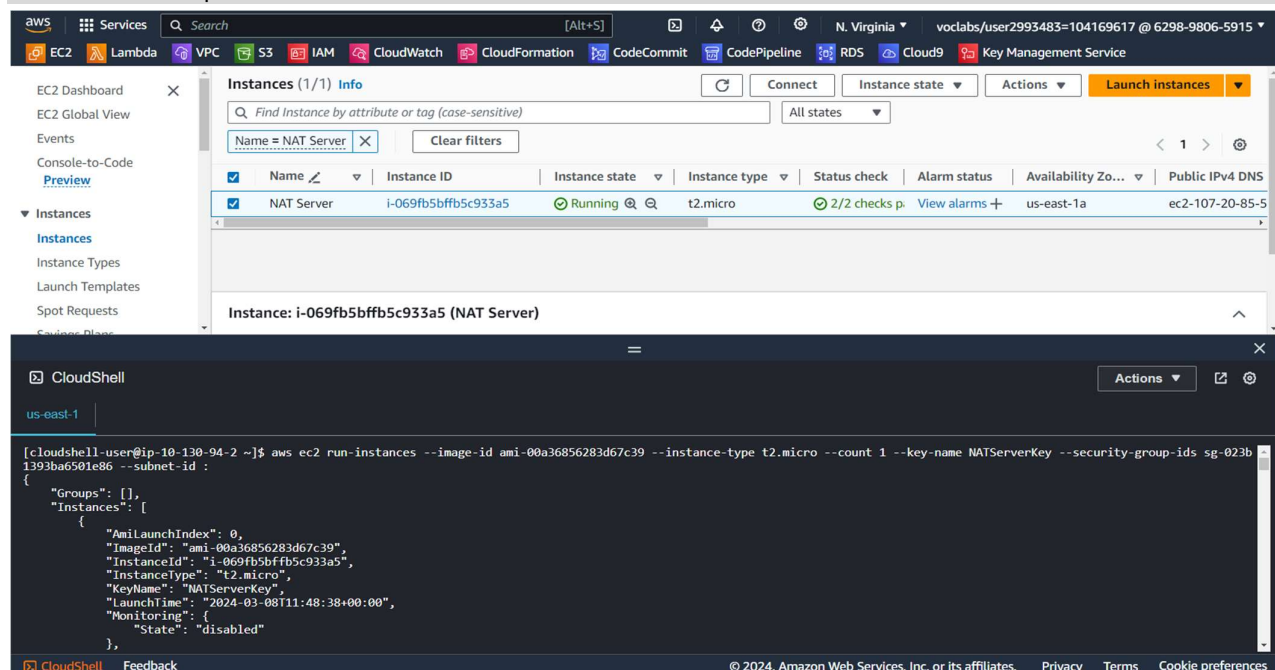
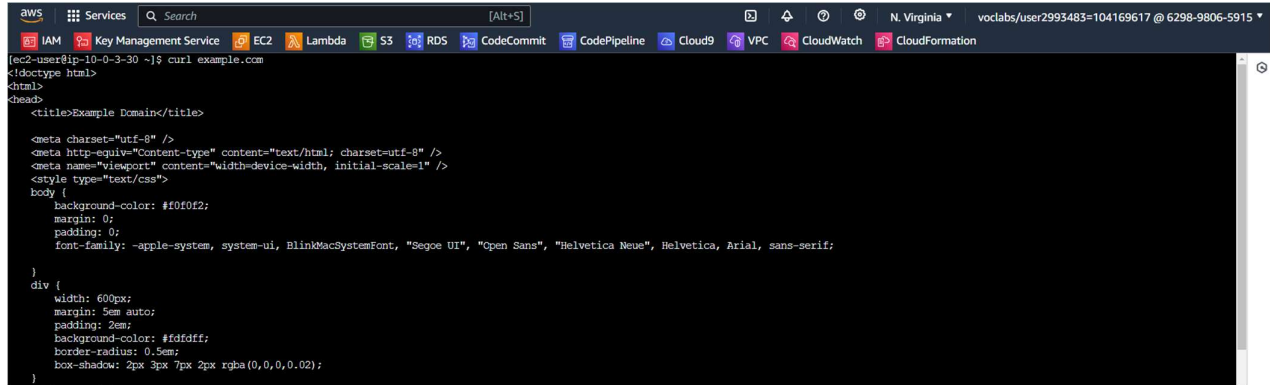


Figure 12 – Launching the NAT Server instance with using CloudShell CLI

Details about the security group **NATServerSG** will be described later. After configuring carefully, we were able to make internet connection from an instance in a private subnet with private IP address **10.0.3.30** (Figure 13).



```

aws
Services
[Alt+S]
N. Virginia
voclabs/user2993483=104169617 @ 6298-9806-5915
IAM Key Management Service EC2 Lambda S3 RDS CodeCommit CodePipeline Cloud9 VPC CloudWatch CloudFormation
[ec2-user@ip-10-0-3-30 ~]$ curl example.com
<doctype html>
<html>
<head>
<title>Example Domain</title>
<meta charset="utf-8" />
<meta http-equiv="Content-type" content="text/html; charset=utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<style type="text/css">
body {
background-color: #f0f0f2;
margin: 0;
padding: 0;
font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
}
div {
width: 600px;
margin: 5em auto;
padding: 2em;
background-color: #fdfdff;
border-radius: 0.5em;
box-shadow: 2px 3px 7px rgba(0,0,0,0.02);
}

```

Figure 13 – SSH into a private instance and using “curl” command to check HTTP connection

E. Auto scaling

In this step, we created an Auto Scaling group that spanned 2 private subnets of **MNguyenVPC**.

Before doing so, we first prepared the necessary custom AMI for the web servers to use. We did that by launching the EC2 instance **Dev Server** in Public Subnet 2. **Dev Server** was based on *Amazon Linux 2 AMI (HVM), SSD Volume Type*, instance type **t2.micro** (Figure 14), assigned to **LabRole** IAM Role and **DevServerSG** security group (Figure 15).

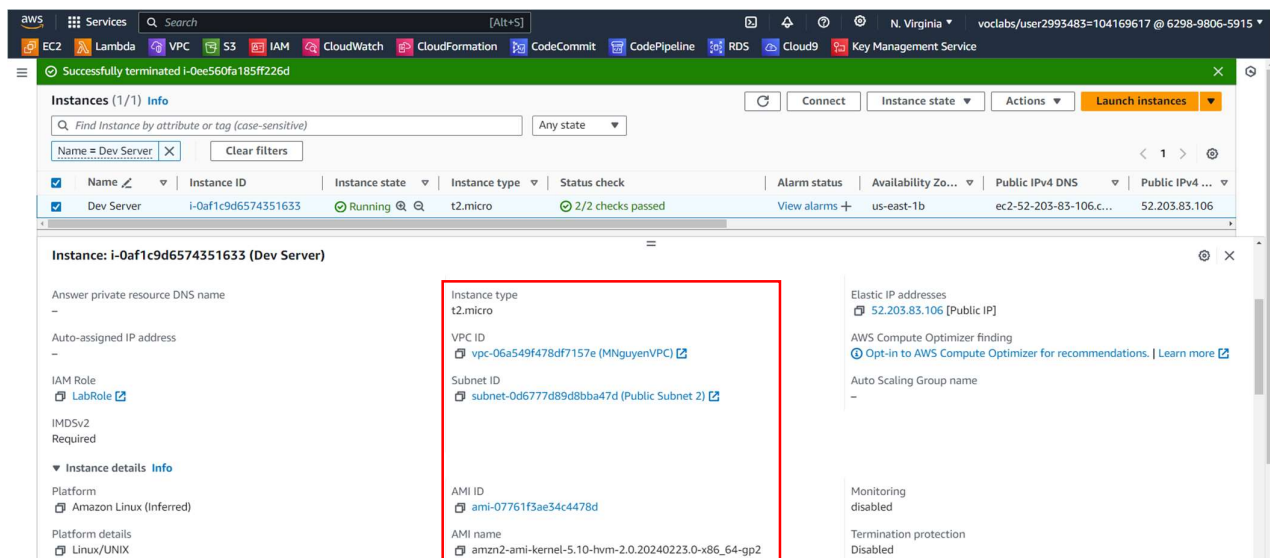


Figure 14 – “Dev Server” instance launched in Public Subnet 2 using Amazon Linux 2 AMI, type t2.micro

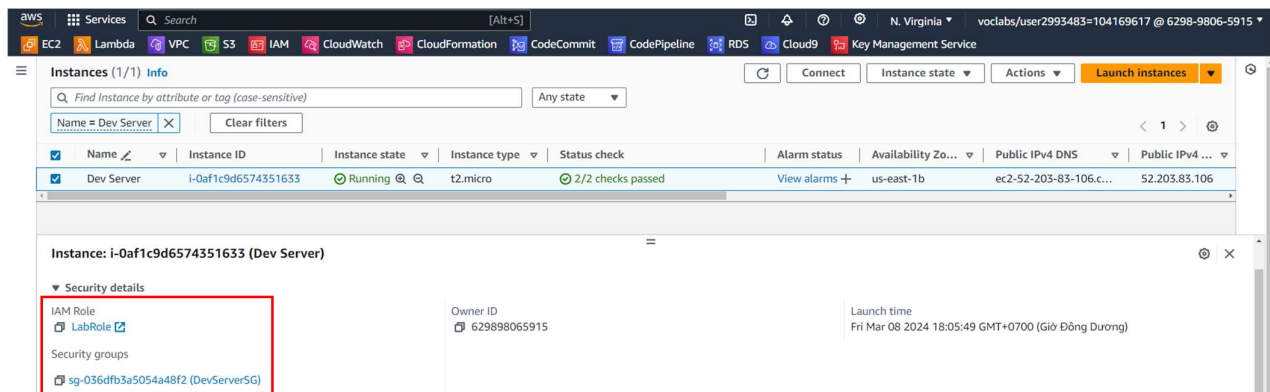


Figure 15 – “Dev Server” had LabRole IAM Role and belonged to DevServerSG security group

We used the same code used for previous assignments from files *Install_PHP_AWS.rtf* and *Install phpMyAdmin on EC2_0.2.pdf* to install Apache web server and phpMyAdmin on **Dev Server**. We also followed the instructions in the *constants.php* file to install AWS SDK for PHP and uploaded the source code for the website provided in *photoalbum.zip* to the instance after making appropriate modifications (Figure 16).

```
aws
[ec2-user@ip-10-0-3-30 ~]$ cd /var/www/html
[ec2-user@ip-10-0-3-30 html]$ ls
aws  aws.zip  photoalbum  phpinfo.php  phpmyadmin  phpMyAdmin-5.2.1-english.zip
[ec2-user@ip-10-0-3-30 html]$
```

Figure 16 – Apache, PHP, phpMyAdmin and AWS SDK were installed, source code was uploaded to “Dev Server” instance

Then, we proceeded to create a custom AMI from the **Dev Server** instance (Figure 17). This took a snapshot of the EBS volume which would be used for future instances using this AMI.

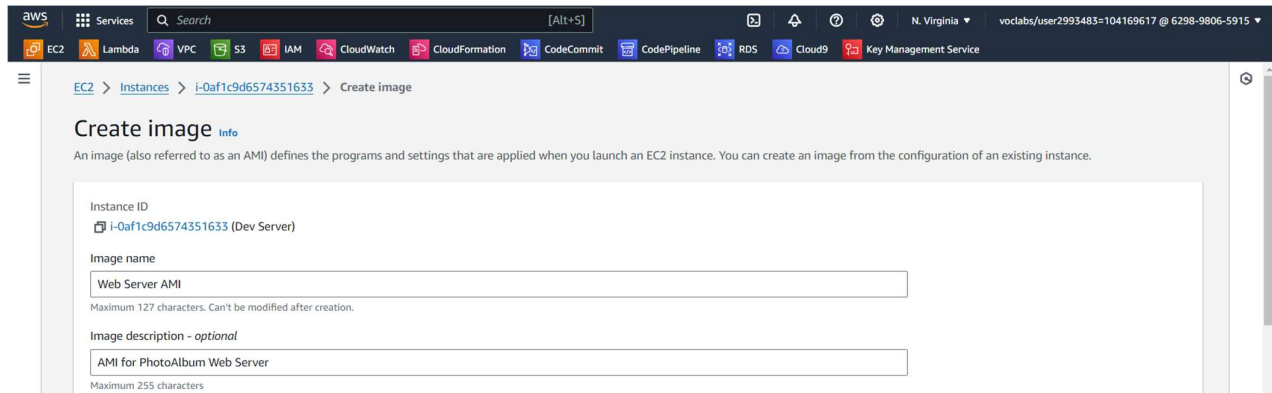


Figure 17 – Creating an AMI “Web Server AMI” from “Dev Server”

Next, we created a Launch Template named **Photoalbum-LT** from **Web Server AMI**, using **WebServerSG** security group and **LabInstanceProfile** IAM instance profile, which would grant the instances **LabRole** IAM role (Figure 18, 19).

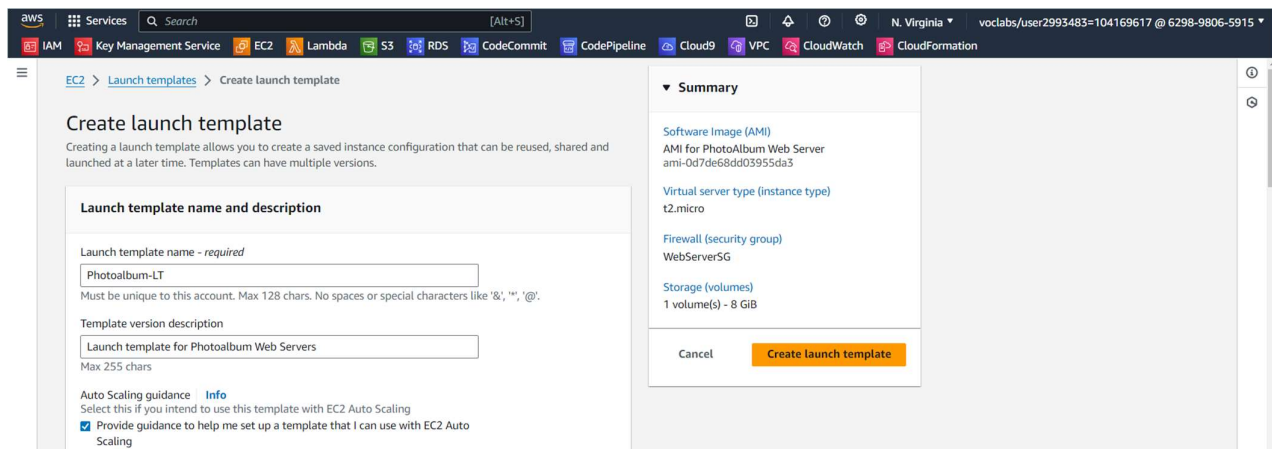


Figure 18 – Creating a launch template from the custom AMI

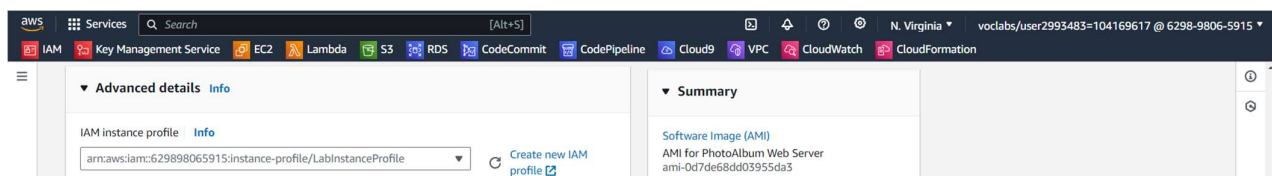


Figure 19 – Choosing “LabInstanceProfile” for IAM instance profile

We moved on and created an Auto Scaling Group (ASG) based on the **Photoalbum-LT** launch template with the name **Photoalbum-ASG** (Figure 20).

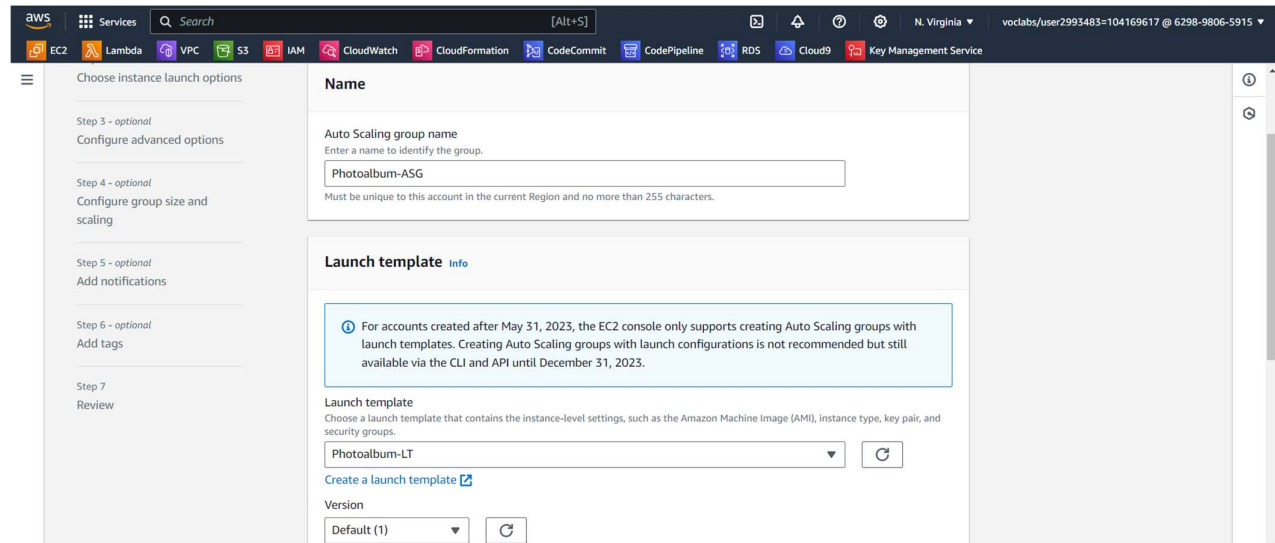


Figure 20 – Creating Auto Scaling group “Photoalbum-ASG” using the newly created launch template

We set the scope of the ASG to private subnets of the VPC, i.e. Private Subnet 1 (10.0.3.0/24) and Private Subnet 2 (10.0.4.0/24), in Step 2 of the ASG creation process (Figure 21).

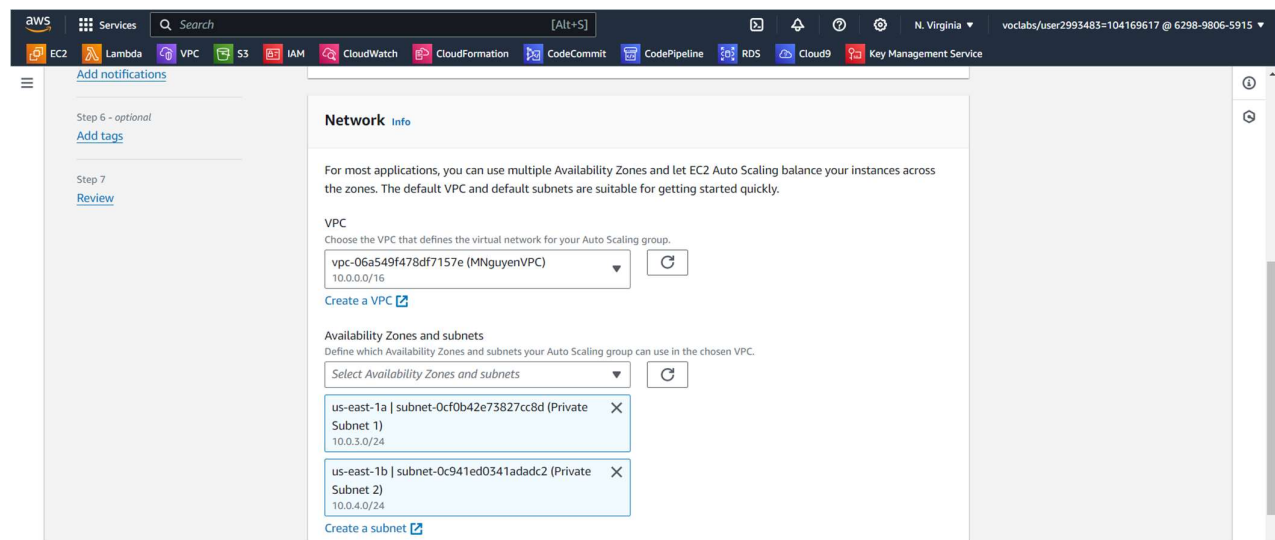


Figure 21 – Mapping the Auto Scaling Group to private subnets in us-east-1a and us-east-1b

Then, in step 3, we attached the ASG to our ALB, **Photoalbum-ELB** with target group **WebServer** using HTTP protocol (Figure 22). In step 4, we defined a scaling policy for **Photoalbum-ASG** to keep the number of servers between 2 and 3, with the **Desired capacity** being 2 (Figure 23). We also configured a target tracking policy to keep the request count per target of **Photoalbum-ELB** target group at 30 for the Auto Scaling group (Figure 24). Finally, we added a tag to name the instances in the ASG **Web Server**.

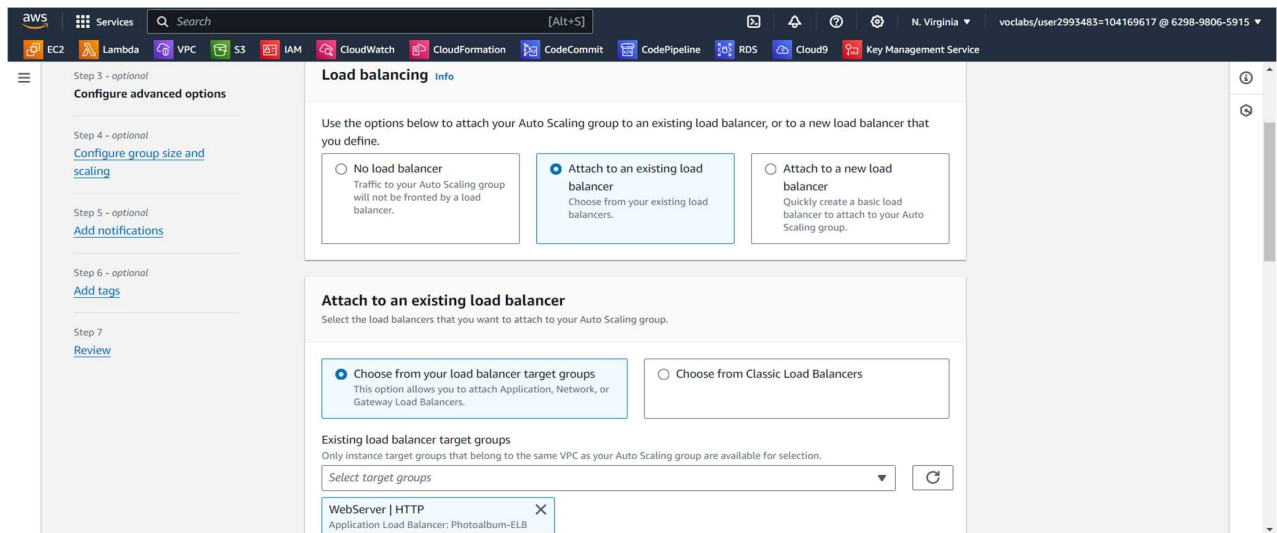


Figure 22 – Attaching the ASG to the ALB with prepared target group and listener

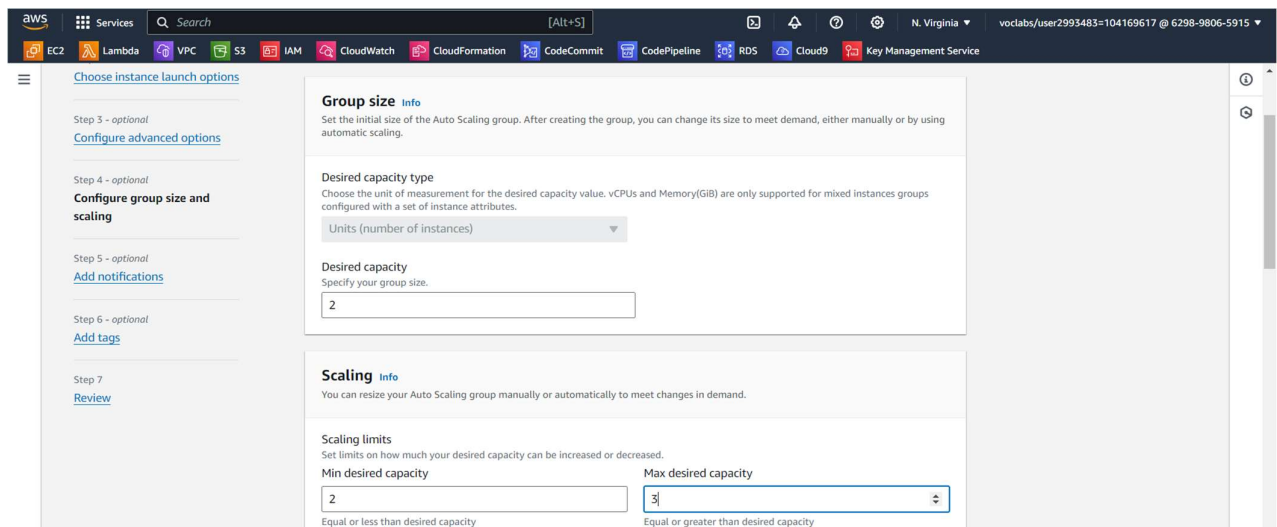


Figure 23 – Setting Desired capacity and Minimum to 2, Maximum to 3

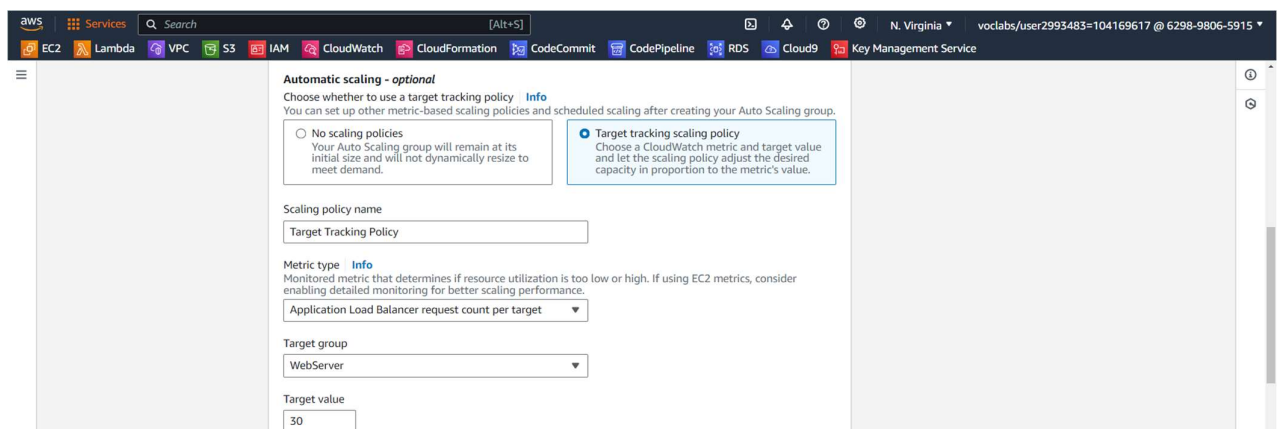


Figure 24 – Choosing a target tracking policy to ensure request count per target of the ALB at 30

F. EC2 instances

After the ASG was successfully created, 2 **Web Server** instances were launched automatically. Figure 25 shows details about one of the servers, in a private subnet, using **Web Server AMI**, and having IAM Role as **LabRole**.

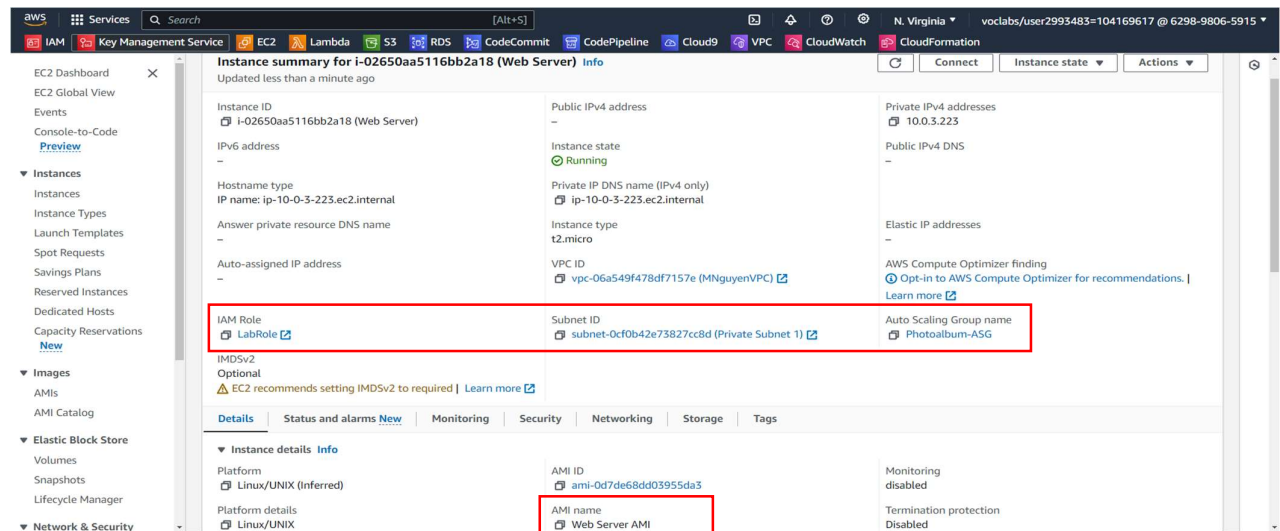


Figure 25 – Details of a Web Server launched automatically by the ASG

G. CreateThumbnail Lambda function

We used **AWS Lambda** to create a function named **CreateThumbnail** and configured it as required, using **LabRole** for **Execution role** to grant access to S3 bucket (Figure 26). Then, we uploaded the provided deployment package (Figure 27).

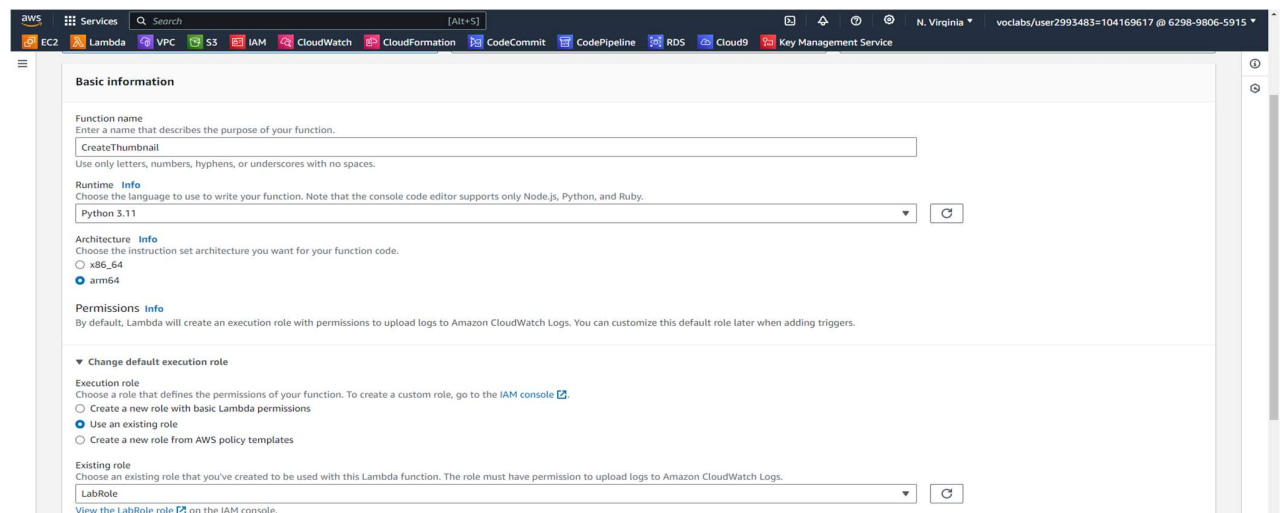


Figure 26 – Creating Lambda function “CreateThumbnail” with necessary configurations

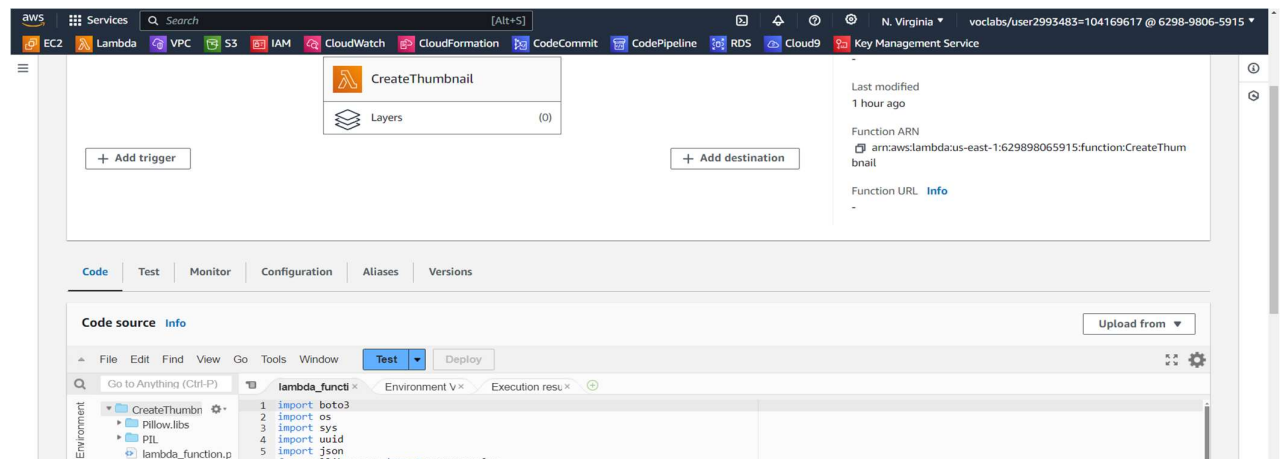


Figure 27 – Uploading the deployment package to the Lambda function

H. Database with RDS

A similar RDS database to that from Assignment 1B was utilized. Figure 28 shows its details.

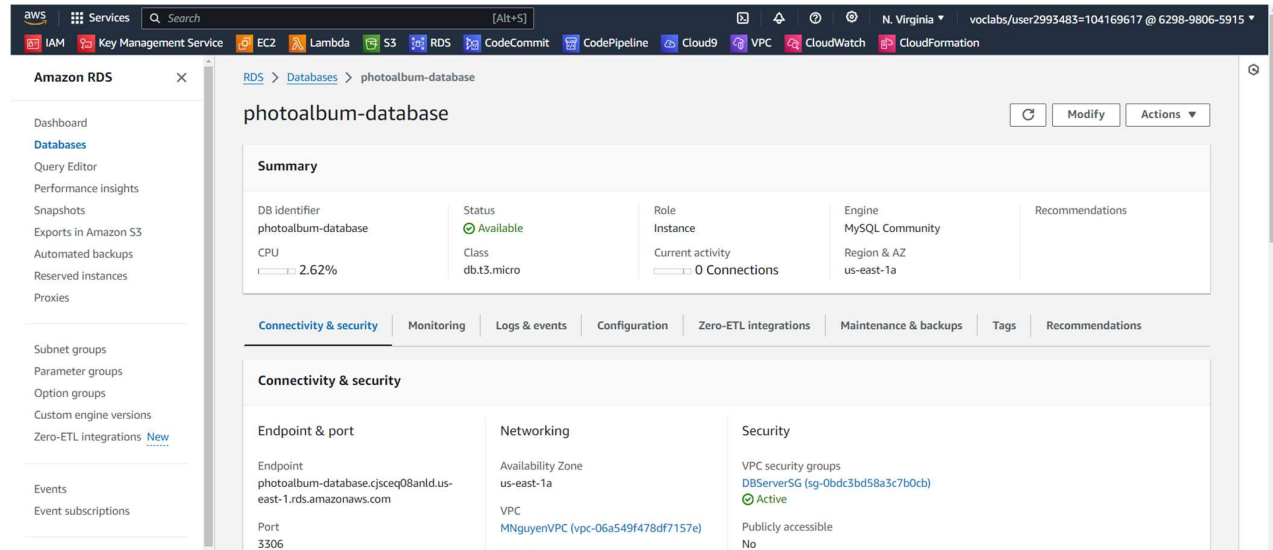


Figure 28 – RDS instance for a MySQL database, with endpoint to be used by phpMyAdmin and photoalbum website

I. Security groups

There were totally 5 security groups in our infrastructure (Figure 29 – 33).

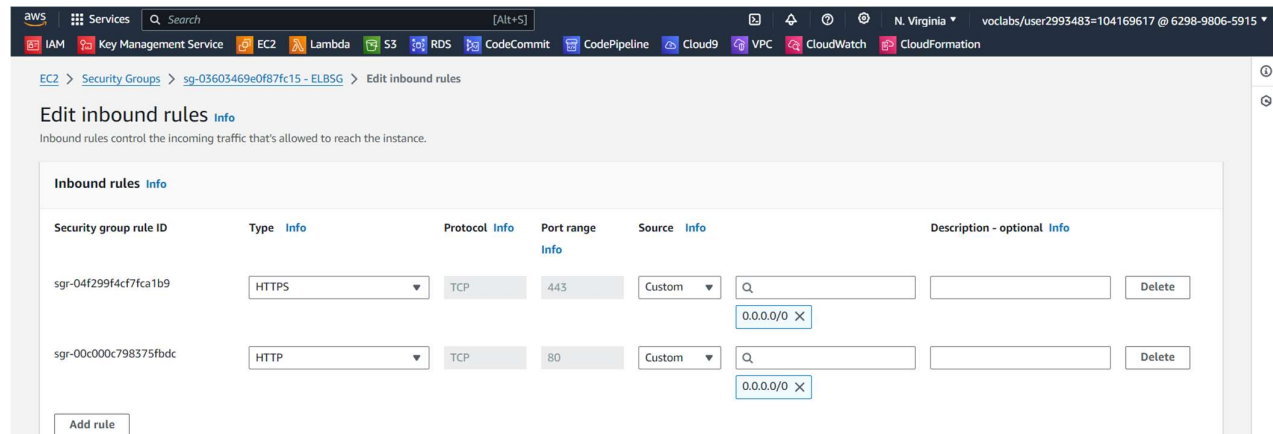


Figure 29 – ELBSG for the ALB Photoalbum-ELB allowing web traffic from anywhere

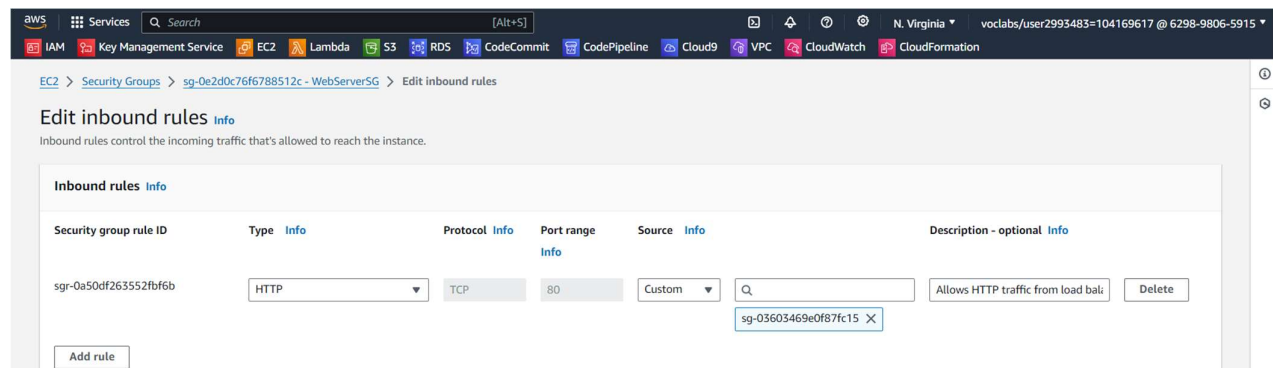


Figure 30 – WebServerSG for Web Servers instances allowing HTTP traffic only from ELBSG

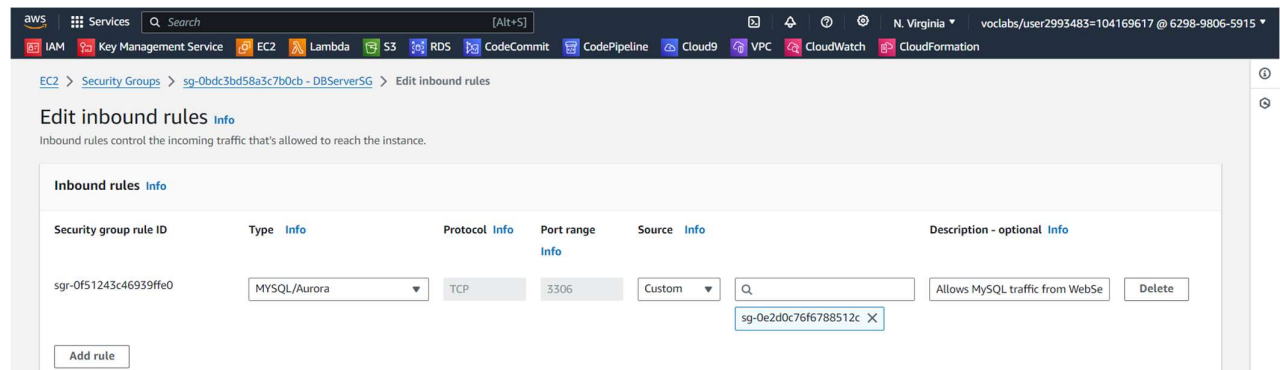


Figure 31 – DBServerSG for the database allowing MySQL traffic from WebServerSG

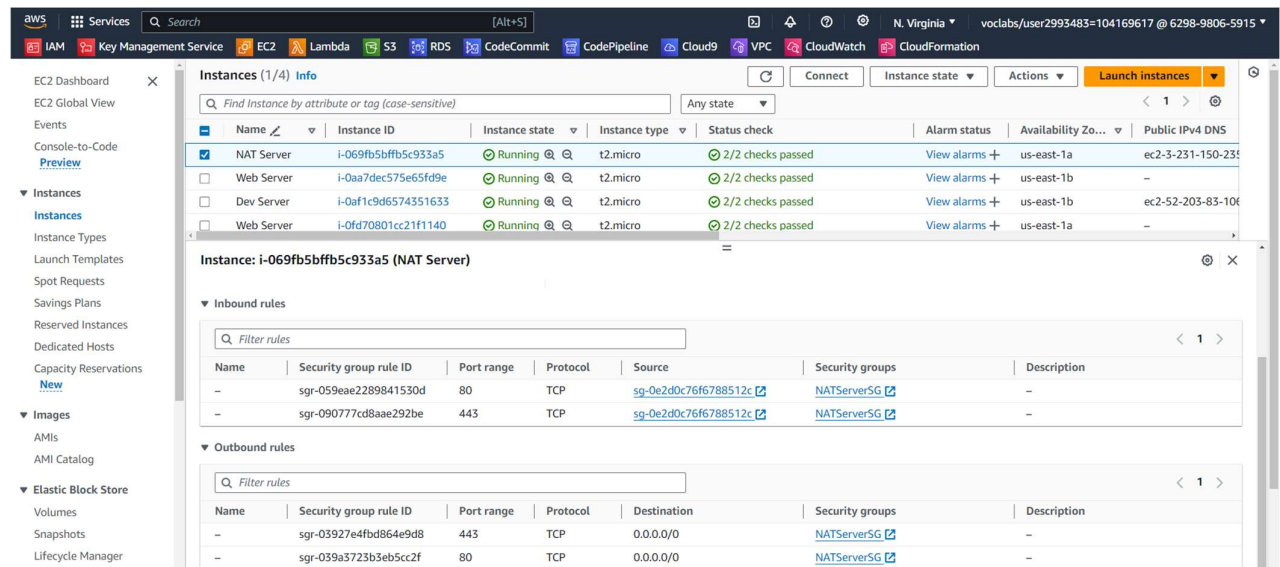


Figure 32 – NATServerSG for the NAT Server instance allowing web traffic from WebServerSG to anywhere

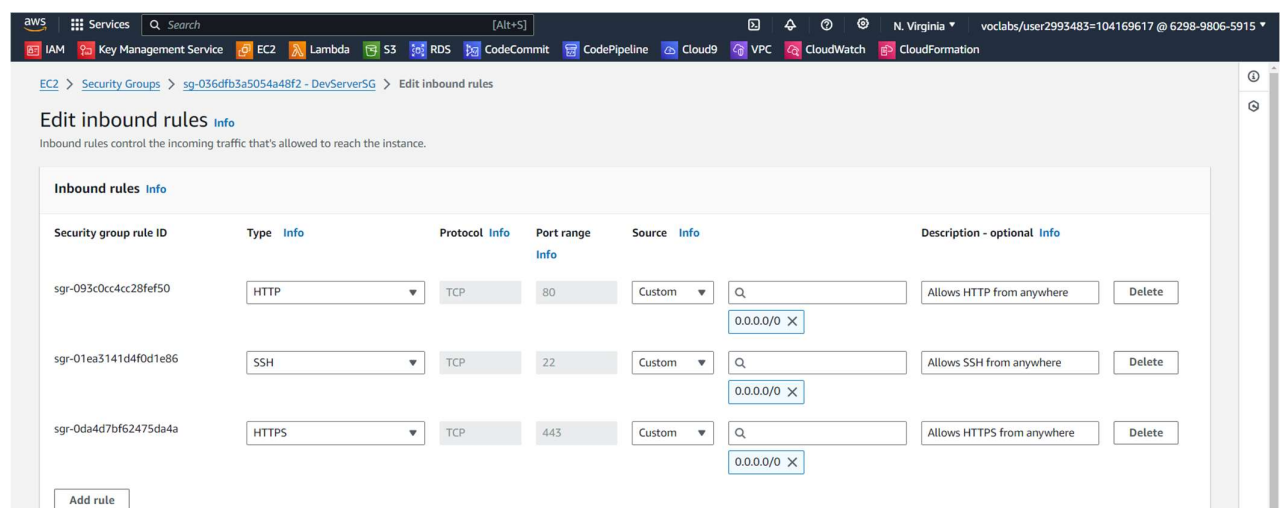


Figure 33 – DevServerSG for the Dev Server instance allowing web and SSH traffic

J. Network ACL

We added a further layer of security to our web servers by deploying a Network ACL named **PrivateSubnetsNACL** to the private subnets to limit traffic in and out the subnets [4]. Most importantly, it blocked ICMP traffic to/from the **Dev Server** instance in Public Subnet 2 (Figure 34, 35).

Specifically, the inbound rules blocked ICMP traffic from Public Subnet 2, and allowed web traffic from within the local network (including the ALB and NAT instance), allowed MySQL traffic from Private Subnet 2 (to reach the database in Private Subnet 1), response traffic from Private Subnet 1 and response traffic from anywhere when web servers make internet connections through NAT (Figure 34).

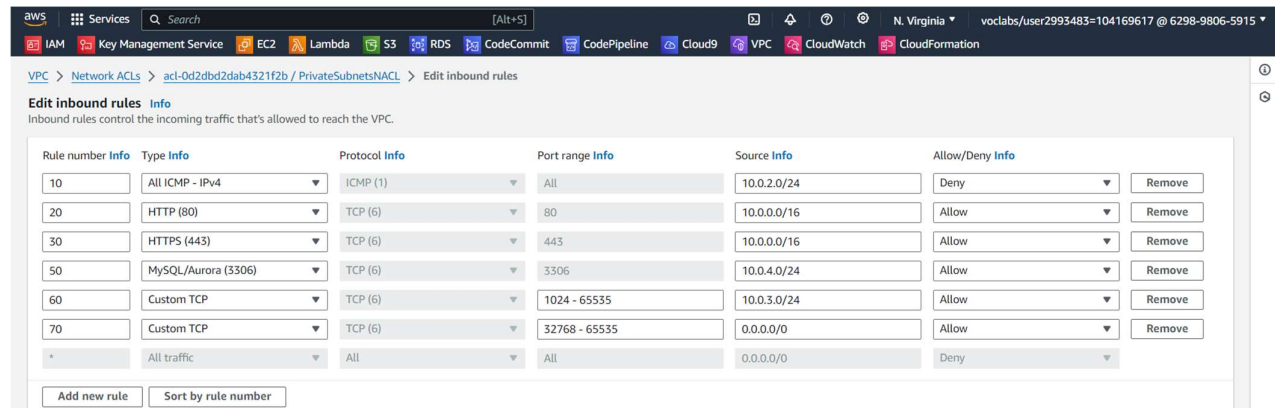


Figure 34 – Inbound rules of “PrivateSubnetsNACL”

Similarly, the NACL blocked ICMP traffic to 10.0.2.0/24, and allowed web traffic to anywhere (for requests from web servers), response traffic (for database requests) to the local network of the VPC, and response web traffic to anywhere (for requests to web servers) (Figure 35).

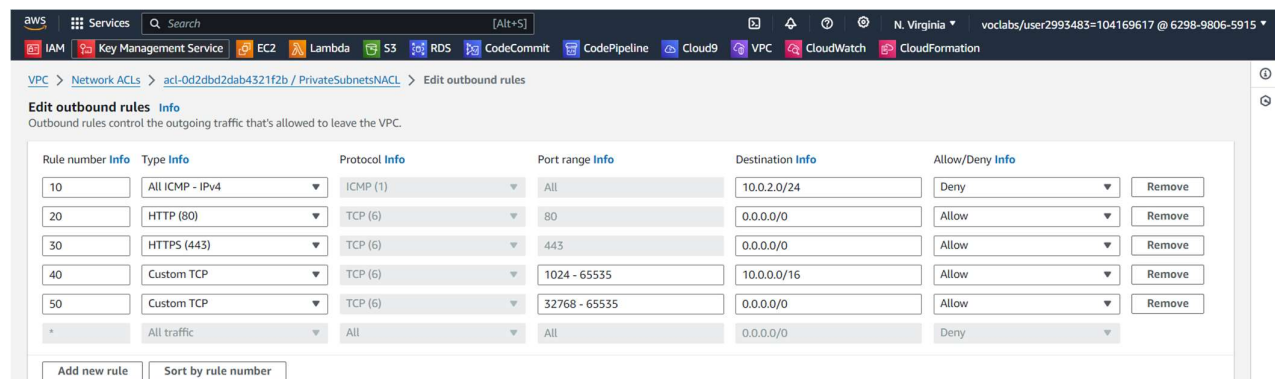


Figure 35 – Outbound rules of “PrivateSubnetsNACL”

III. TESTING

We uploaded some photos to the database and the S3 bucket (including thumbnail generated by the Lambda function) using the URL: <http://photoalbum-elb-336195294.us-east-1.elb.amazonaws.com/photoalbum/photouploader.php> (Figure 36).

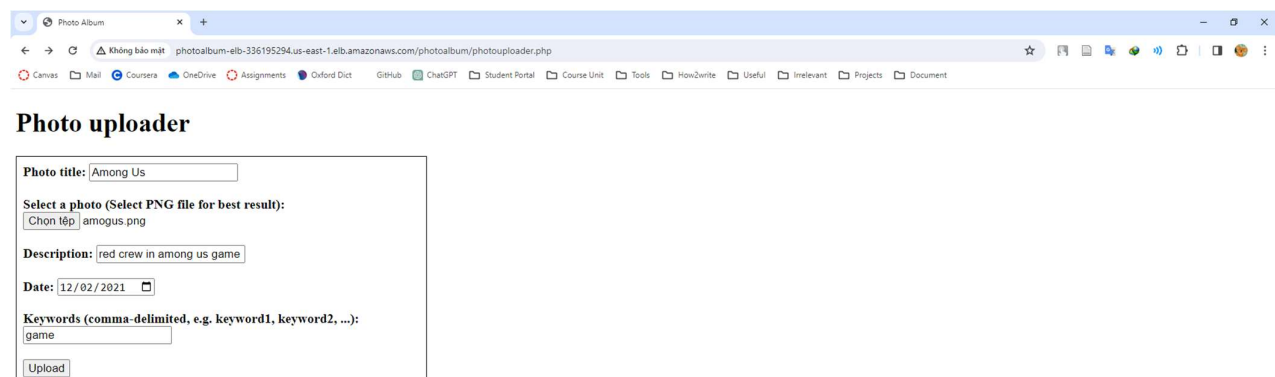


Figure 36 – Uploading a PNG image to the website served by the Load Balancer

The PhotoAlbum website was served with high availability, and accessible only through the DNS name of the Load Balancer: <http://photoalbum-elb-336195294.us-east-1.elb.amazonaws.com/photoalbum/album.php> (Figure 37). We confirmed that the image was uploaded to the S3 bucket (Figure 38), and its meta-data was inserted to the database (Figure 39).

Student name: Nguyen Thi Thanh Minh
 Student ID: 104169617
 Tutorial session: Monday 01:00PM
 Instance ID: i-060996cdd966af92; Availability Zone: us-east-1a

Uploaded photos:
[Upload more photos](#)

Photo	Name	Description	Creation date	Keywords	Thumbnail
	Swinburne Logo	Logo of Swinburne University	2024-03-06	university, logo, australia	
	Koala	yellow koala	2024-03-01	animal, australia	
	Among Us	red crew in among us game	2021-02-12	game	

The browser's developer console shows the HTML structure of the photo table, with the thumbnail image source highlighted in red:

```

<td>
  
</td>
<td>Among Us</td>
<td>red crew in among us game</td>
<td>2021-02-12</td>
<td>game</td>
<td>
  
</td>
</tr>
</tbody>
</table>
  
```

Figure 37 – album.php served by the ALB, from Web Server in us-east-1a (we modified the code to show instance meta-data and the thumbnail image generated by Lambda)

photoalbum-s3bucket

Objects (10) Info

Name	Type	Last modified	Size	Storage class
amogus.png	png	March 8, 2024, 20:01:53 (UTC+07:00)	4.3 KB	Standard
Bennett.png	png	March 9, 2024, 16:39:47 (UTC+07:00)	678.2 KB	Standard
koala.png	png	March 8, 2024, 13:54:21 (UTC+07:00)	141.2 KB	Standard
resized-amogus.png	png	March 8, 2024, 20:01:55 (UTC+07:00)	1.8 KB	Standard

Figure 38 – Both the original image file and its thumbnail (generated by Lambda) were present in the S3 bucket

Showing rows 0 - 2 (3 total. Query took 0.0005 seconds)

```

SELECT * FROM `photo_metadata`
  
```

id	title	description	creationdate	keywords	reference
1	Swinburne Logo	Logo of Swinburne University	2024-03-06	university, logo, australia	https://photoalbum-s3bucket.s3.amazonaws.com/Swinb...
2	Koala	yellow koala	2024-03-01	animal, australia	https://photoalbum-s3bucket.s3.amazonaws.com/koala...
3	Among Us	red crew in among us game	2021-02-12	game	https://photoalbum-s3bucket.s3.amazonaws.com/amogu...

Figure 39 – The image's meta-data was inserted as a record to the database (notice the phpMyAdmin was accessible using the ALB thanks to Stickiness attribute of target group)

We also confirmed that all EC2 targets were healthy, and they would be replaced automatically by the ASG when terminated (Figure 40). Moreover, we checked that without using the ALB's DNS name, the images in S3 bucket would not be accessible (Figure 41). Finally, we tested the NACL functionality by pinging a web server from the Dev server (Figure 42).

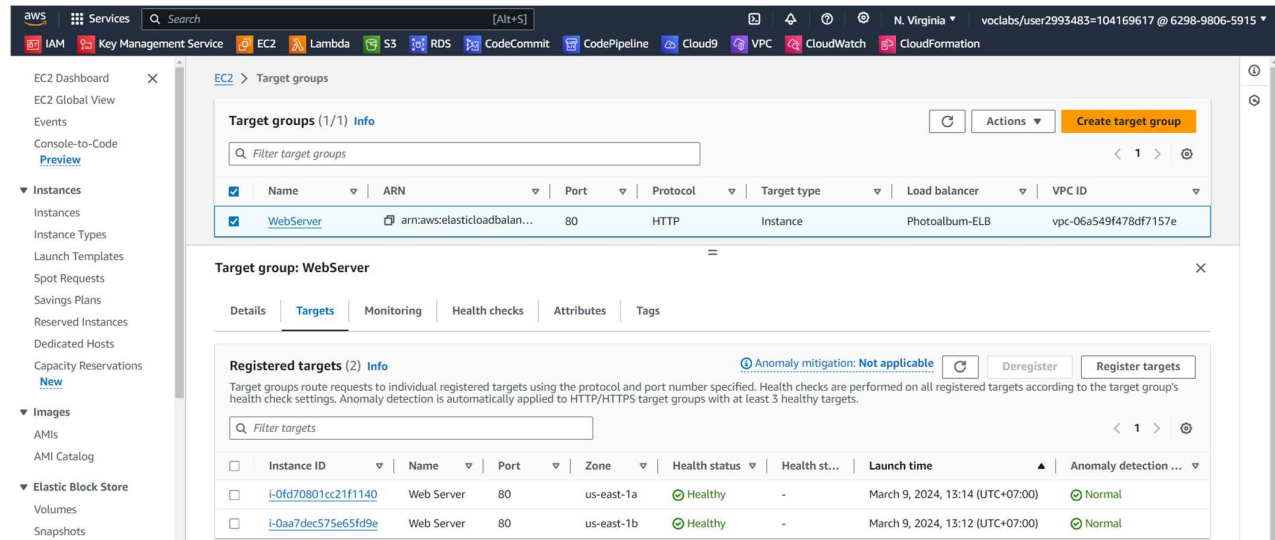


Figure 40 – All target instances had “Healthy” Health status because they passed health checks by the ALB



Figure 41 – Images in the S3 bucket were not accessible when using direct URL

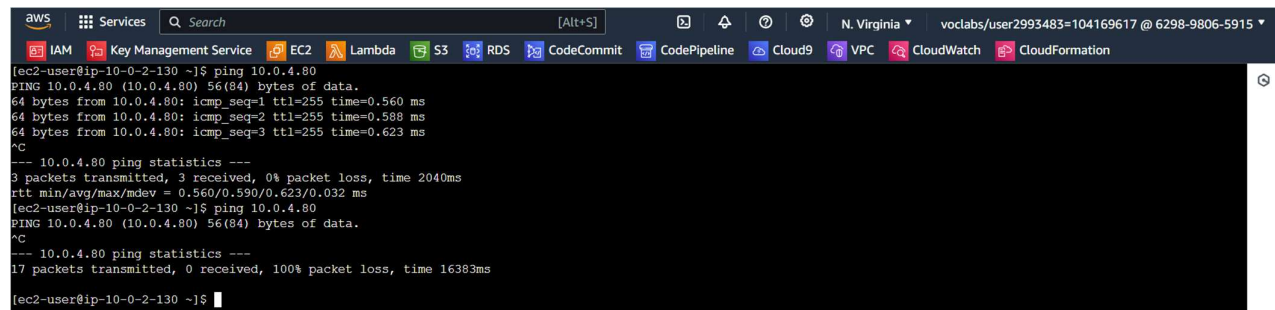


Figure 42 – Pinging a Web Server from Dev Server, without NACL (first attempt succeeded) and with NACL (second attempt failed)

IV. REFERENCES

- [1] “Bucket policy examples - Amazon Simple Storage Service.” Accessed: Mar. 08, 2024. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/example-bucket-policies.html#example-bucket-policies-folders>
- [2] “Sticky sessions for your Application Load Balancer - Elastic Load Balancing.” Accessed: Mar. 08, 2024. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/sticky-sessions.html>
- [3] “NAT instances - Amazon Virtual Private Cloud.” Accessed: Mar. 08, 2024. [Online]. Available: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_NAT_Instance.html
- [4] “Control traffic to subnets using network ACLs - Amazon Virtual Private Cloud.” Accessed: Mar. 08, 2024. [Online]. Available: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

The End